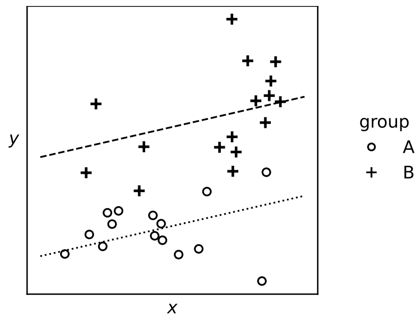
COMPUTER SCIENCE TRIPOS Part IB – 2024 – Paper 6

6 Data Science (djw1005)



This plot shows a collection of datapoints $(x_i, y_i, group_i)$ from two groups, together with fitted lines for each group. Based on our scientific understanding of the dataset, the lines should be parallel.

linear model

(*a*) Describe a probability model and a fitting procedure that might be behind this plot. Explain how you enforce the 'parallel lines' requirement. Give pseudocode for the fitting procedure. Explain briefly how the plot is produced. [*Note:* When describing a probability model, you should state your assumptions: what probability distributions are you using, and why have you chosen them?]
[7 marks]

Bayesianism, confidence intervals

(*b*) We would like to report our confidence about the offset between the two groups. Explain the Bayesian approach to computing this confidence, and give pseudocode. [7 marks]

Bayesianism, prediction

(*c*) We have been asked to predict y at some new value x^*, for both groups. Explain how to compute confidence intervals for these two predictions, and give pseudocode. [6 marks]

Answer:

Part (a). Consider the probability model

$$Y_i \sim \text{Normal}(\alpha + \beta x_i + \delta 1_{group_i=1}, \sigma^2)$$

where α, β, δ, and σ are unknown parameters. We can fit this to the dataset using maximum likelihood estimation; the estimates for the first three parameters correspond to a least squares estimate of the linear model

$$\mathbf{y} \approx \alpha + \beta\mathbf{x} + \delta 1_{\mathbf{group}=B}$$

where bold symbols denote vectors. To fit it, assuming my data is in a dataframe `df`,

```
model0 = sklearn.linear_model.LinearRegression()
def X0(x,g): return np.column_stack([x, np.where(g=='B',1,0)])
model0.fit(X0(df.x, df.group), df.y)
```

The plot shows predicted responses at evenly-spaced values on the horizontal axis, which we can obtain with

```
a,(b,d) = model0.intercept_, model0.coef_
xnew = np.linspace(0,10,100)
ypred_group0 = a + b*xnew
ypred_group1 = a + b*xnew + d
```

Marks: 2 for the probability model, 2 for the fitting procedure, 3 for the code including `ypred`*. The code here is valid Python, but any pseudocode is acceptable. An answer based on plain maximum likelihood estimation and an optimization routine such as* `scipy.optimize.fmin` *is also acceptable. An answer using only low-level routines is acceptable. The answer may be presented as a script, or wrapped up as a function.*

Part (b). We'll assume we also have some prior belief about the four unknown parameters. It's impossible to come up with numbers given that the plot doesn't have a scale, but we might choose something like $\alpha \sim N(0,1)$, $\beta \sim N(0,1)$, $\delta \sim N(0,1)$, $\sigma \sim \text{Exp}(1)$, all four independent. What's important is that these prior distributions should be chosen from background scientific understanding, not from looking at the data.

The confidence procedure is as follows. First, generate many samples $(\alpha_j, \beta_j, \delta_j, \sigma_j)$ from the prior. Next, compute weights and rescale them to sum to one,

```
def pr(y,a,b,d,s,x,g):
  return scipy.stats.norm.pdf(y, loc=a+b*x+d*(1 if g=='B' else 0), scale=s)
w = [np.prod(pr(y,a,b,d,s,x,g) for y,x,g in dataset)
     for (a,b,d,s) in prior_samples]
w = w / np.sum(w)
```

In algebra the first line defining `w` corresponds to

$$w_j = \prod_{i=1}^{n} \Pr(y_i \mid \alpha_j, \beta_j, \delta_j, \sigma_j \; ; \; x_i, group_i)$$

where the likelihood is for a Gaussian with the appropriate mean, i.e.

$$\frac{1}{\sqrt{2\pi\sigma_j^2}} \exp\Big\{-\frac{1}{2\sigma_j^2}\big(y_i - \alpha_j - \beta_j x_i - \delta_j 1_{group_i=B}\big)^2\Big\}.$$

Finally, use the sampled δ_j and associated weights w_j to obtain a 95% confidence interval for δ:

```
d_ = [d for (a,b,d,s) in prior_samples]
i = np.argsort(d)
ws, d_s = w[i], d_[i]
F = np.cumsum(ws)
(lo,hi) = (d_s[F<0.025][-1], d_s[F>-.975][0])
```

```
f"A 95% confidence interval for delta is [{lo},{hi}]"
```

This is a computational approximation to

$$\mathbb{P}(\delta \in [\mathrm{lo}, \mathrm{hi}] \mid \mathrm{data}) = 95\%.$$

Marks: 2 for the prior, 2 for the weights, 3 for the confidence interval. The code here is valid Python, but any pseudocode is acceptable. The question doesn't ask for algebra, but it's given for the benefit of student revision.

Part (c). For group B, we compute predicted values under each of the sampled parameters:

```
pred_ = [a+b*xstar+d for (a,b,d,s) in prior_samples]
```

and then we compute a confidence interval using the same code as above, but applied to `pred_` rather than to `d_`. For group A, the same thing but without the `+d`. *Marks: 4 for the vectors of predictions, 2 for applying the confint procedure*